

ft Saurabh Deep-learning Geheai me Padhna

Page No.:

Date:

⇒ Pehla Saurabh ?? (ANN)

What are weights, inputs and bias?

⇒ What is a perceptron?

→ It's a single layer neural network.

• It's a basic unit of ANN

Let's say I have a dataset :-

x_1 IQ	x_2 Study hours	O/P Pass or Fail
95	3	0
111	4	1
101	5	1

Now let's understand these concepts from the eg

→ Input Layer

→ Hidden Layer

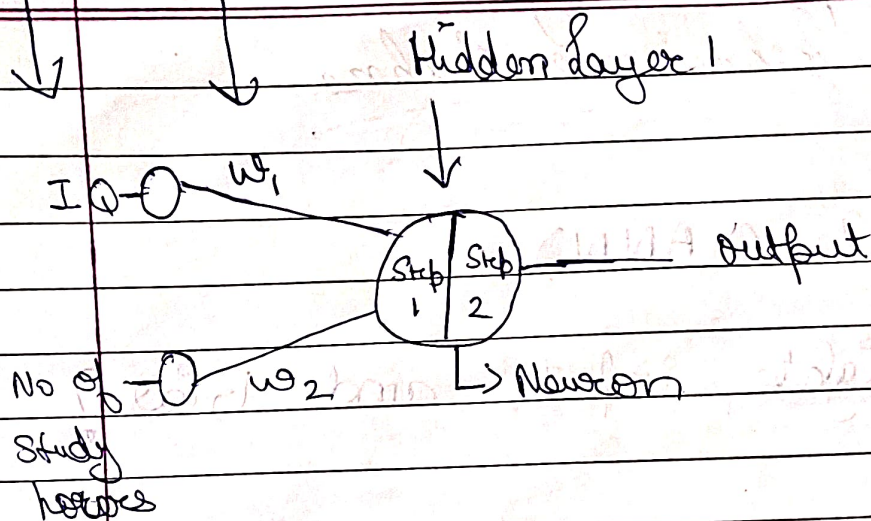
→ Weights

→ Activation Functions

Input Weights (w_1, w_2)

Page No.:

Date:



Why do we call it a single layer NN?

Step - 1

$$Z = w_1 x_1 + w_2 x_2 + b$$
$$Z = \sum_{i=1}^n x_i w_i + b$$

bias creates a noise or say an intercept

Step - 2 We apply activation function.

What is an activation function?

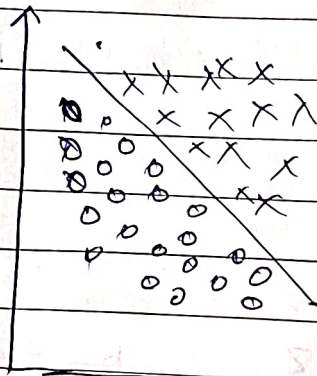
→ Transform the output to desired range.

→ Finally we get an output say 0 or 1

Now I check the error with actual record & accordingly will change or update the weights if needed.

Now this single-layer perceptron is used for binary classification. It can't be used for multi-label classification. We need a multi-layer perceptron for this.

So what do we remember?



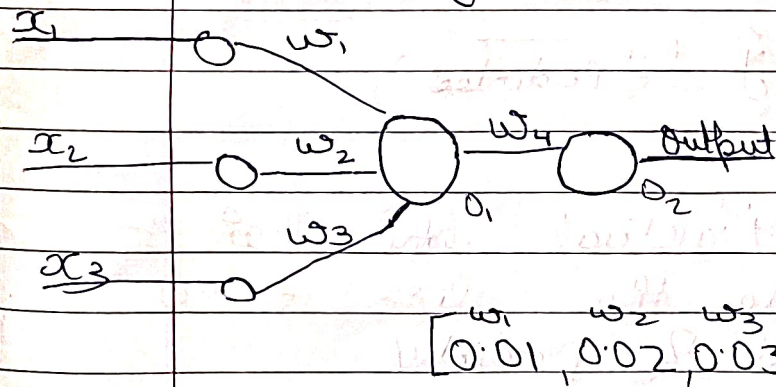
Perceptron is a ~~binary~~ ^{Linear} classifier.

Multilayered Perceptron Model

Simple jaisa pehle buja ye sab
for used prop agation me the

lets see 1 more eg then use
will jump to backpropagation.

Forward Propagation



<u>IQ</u>	<u>Study hour</u>	<u>Play hour</u>	<u>Pass/Fail</u>
95	4	4	1
100	5	2	1
95	2	1	0

$$[0.01, 0.02, 0.03] [0.001]$$

Step 1

$$Z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$= 1.151$$

Step -2 Activation (Sigmoid)

$$= \frac{1}{1 + e^{-z}}$$

$$O_1 = \frac{1}{1 + e^{-1.151}} = 0.759$$

Hidden Layer - 2 $w_4 = 0.02$ $b_2 = 0.03$

Step - 1: $z = O_1 \times w_4 + b_2$

$$= 0.759 \times 0.02 + 0.03$$

Step - 2 activation (Sigmoid)

$$= \frac{1}{1 + e^{-z}}$$

$$O_2 = \frac{1}{1 + e^{-0.0458}}$$

$$= 0.51129 = \hat{y} \text{ (y predicted)}$$

As $\hat{y}$ milga and y actual hai so we would calculate the loss to see diff btwn actual & predicted

$$\text{Loss } \text{MSE} = (y - \hat{y})^2$$

$$= (1 - 0.51129)^2$$

$$= 0.41 \downarrow \downarrow \downarrow \text{ (Very poor)}$$

Now There are different types of Activation function which are used in diff scenarios.

⇒ Step activation

Formula

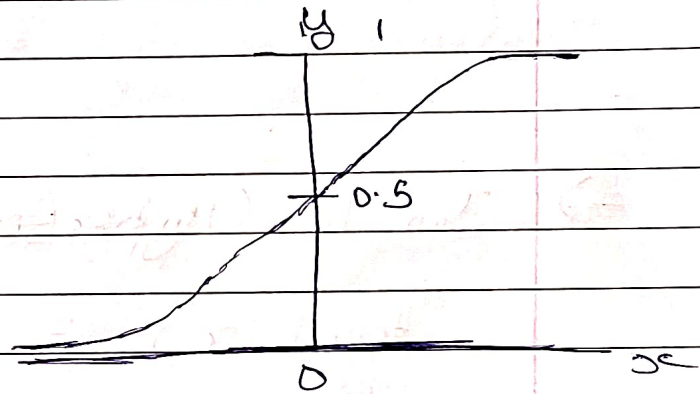
$$f(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$$

⇒ Not mostly used nowadays.

2.) Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Range :- 0 to 1

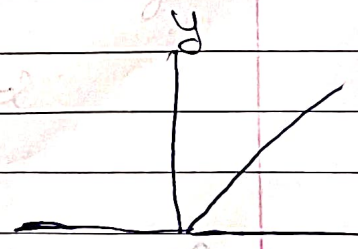


Used mostly for binary classification.

3.) Relu

$$\text{Relu}(x) = \max(0, x)$$

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$$

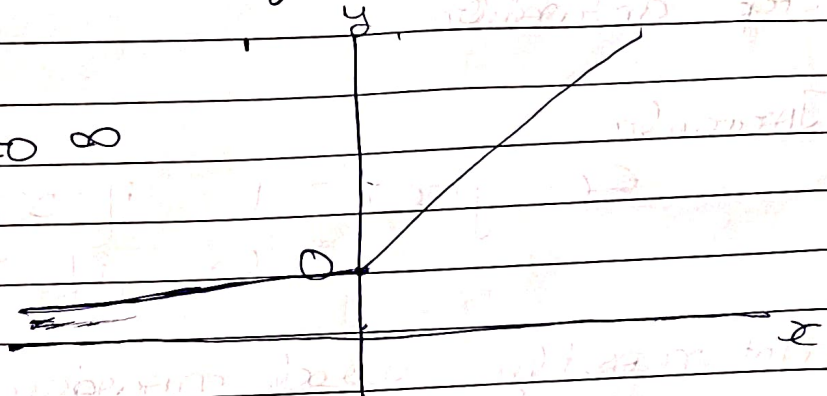


Range :- 0 to ∞

⇒ Leaky ReLU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{if } x < 0 \end{cases}$$

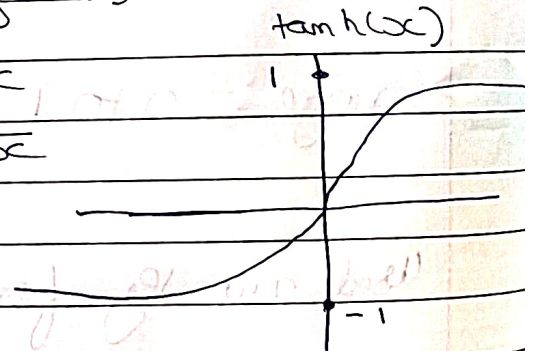
Range: $-\infty$ to ∞



Used when ReLU fails

⇒ Tanh (Hyperbolic Tangent)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Range: -1 to 1

Used in hidden layers when data is centered around 0

⇒ Softmax (IMP)

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Range: 0 to 1

Used in Multi-Class Classification

Now Predicted $\hat{y}$ is the sum log

Also we calculated $\hat{y}$ we now get the loss function for calculating the loss

1.) M.S.E (Mean Squared Error)

Formula (Regression Loss)

$$M.S.E = \frac{1}{N} \sum (y - \hat{y})^2$$

Used while predicting continuous values

Eg: House Price Prediction

2.) Binary Cross Entropy Loss (Classification)

$$L = -\frac{1}{N} \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

Used when output is probability

Eg: Cancer detection

Now we got the error. To minimize the error we update the weights through backword propagation.

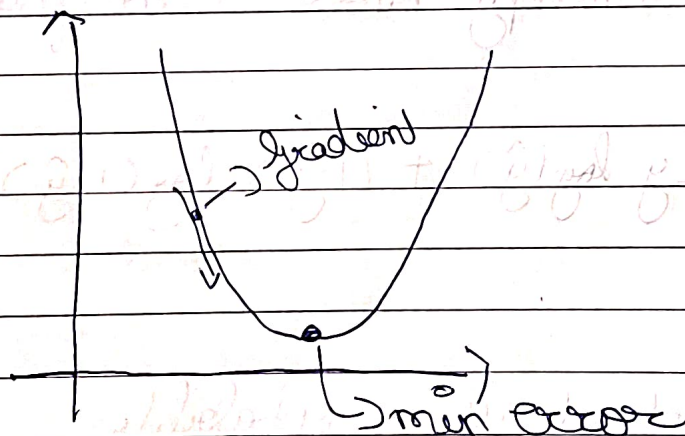
- Gradient descent update

$$w_{\text{new}} =$$

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}$$

- ~~There is a~~ $\eta = \text{learning rate}$

$$\frac{\partial L}{\partial w} = \text{gradient}$$



$\Rightarrow$ How do we get this $\frac{\partial L}{\partial w_i}$

$\Rightarrow$ The dependency change uses

$$w_0 \rightarrow z_0 \rightarrow \hat{y} \rightarrow L$$

So applying chain Rule.

$$\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_0} \cdot \frac{\partial z_0}{\partial w_0}$$

This is the entire output layer gradient

Now

$$\frac{\partial L}{\partial \hat{y}} = (\hat{y} - y)$$

$$\frac{\partial \hat{y}}{\partial z_0} = \hat{y} (1 - \hat{y})$$

$$\frac{\partial z_0}{\partial w_0} = 1 \text{ or } h \quad (\text{The output we get from 1st neuron})$$

$$\therefore \boxed{\frac{\partial L}{\partial w_0} = (\hat{y} - y) (\hat{y}) (1 - \hat{y}) (h)}$$

Now if this slope is +ve we decrease the weights and if slope is negative we increase the weight.